

Programmering med mikrokontroller

Aut2602 Innlevering 5

Jens Christian Valen Leynse

Oktober 2024

Skoleeksamen i: AUT-2602 Programmering med mikrokontroller

Dato og tidspunkt: Klokka 09:00–13:00, 22.11.2024

Målform: Bokmål

Fagansvarlig (Navn og telefonnr.): Per Anton Øverseth Olsen

Antall sider: 6 inkludert forside

Support: Du kan ringe 776 20 880 for support på eksamensdagen

Vekting av spørsmål, eller annen informasjon: Besvarelsen kan være på norsk, engelsk eller en blanding.

Viktig informasjon om sitering og plagiering:

1. Dette er en individuell eksamen som skal besvares uten samarbeid med andre.
2. Alle hjelpemidler er tillatt (egne notater, pdfer fra forelesningene, lærebok, internett etc. Se Canvas eller uit.no for info om KI).
3. Alle eksamener som leveres i WISEFLOW blir automatisk sjekket for plagiat. Det er ikke tillatt å kopiere medstudenter, nettressurser, kilder, eller litteratur uten referanser.

Oppgave 1: Faktaspørsmål

I denne oppgaven skal dere svare så kort som mulig. Dersom det spørres om pinner, svar med PA0 for PORTA, pinne 0. Liten n indikerer heltall, og liten x indikerer bokstav, slik som PORTx.PINnCTRL.

- a) 4% Hva menes med “full duplex” og “half duplex” i USART-kommunikasjon med og uten RS485-funksjonalitet?
- b) 4% Hva må stå i registeret “USARTn.BAUD” dersom baudrate skal være på 9600, i asynchronous normal mode, ordlengden er på 8-bit og CLK_PER = 12MHz?

- c) 4% Mange periferi-moduler på AVR128DB har registrene INTCTRL og INTFLAGS, med korresponderende bits (slik som OVF_bp). Hva er sammenhengen mellom disse to registertypene?
- d) 4% Hvilke pinner kan benyttes til 8-bit-PWM dersom man bruker TCA0 på AVR128DB48?
- e) 4% Hva er forskjellen på å implementere regelmessig ADC-lesing med og uten å bruke event-routing (EVSYS) og hvordan påvirker dette sleep-funksjonalitet?
- f) 3% Hva er grunnen for at det anbefales å benytte pull-up på ubrukte pinner dersom man vil spare strøm på en mikrokontroller?
- g) 5% Hvilke faktorer er viktige å tenke på dersom CLK_PER endres for et innebygd system med digitale og analoge sensorer tilkoblet?
- h) 4% Beskriv kort hvordan en AVR128DB kan lese av en verdi fra en tenkt ekstern sensor ved hjelp av SPI.
- i) 4% Forklar hovedforskjellen mellom å implementere ADC med single conversion mode, og free-running mode, hvis programavbrudd benyttes i begge tilfeller.
- j) 5% SW1 er en typisk trykk-knapp. Hva er hensikten med en kondensatoren C1 og motstanden Rp1 i figur 1, og hvordan kan man med kode tilnærme seg samme effekt i mangel på kondensator?
(Figur 1: Krets med knapp tilkoblet en pinne.)
- k) 5% Et program har mange forskjellige oppgaver som skal gjøres regelmessig. Hva er forskjellen mellom å bruke `_delay_ms()`, RTC med “overflow interrupt” og ISR-funksjon og TCA med “overflow interrupt” og ISR-funksjon?

Oppgave 2: Kodeforståelse og feilsøking

a) 7%

Beregn de manglende verdiene og forklar kodelinjene merket med ? i kodeut-draget nedenfor, slik at tonen som høres på buzzeren øker gradvis fra 500Hz til 1kHz på 4 sekunder og repeterer dette. Du kan også foreslå endringer hvis det er behov.

```
#include <avr.io.h>
#include <avr/interrupt.h>
#define F_CPU 4000000UL
#define BUZZER_PORT PORTF
#define BUZZER_PIN_bp 2
volatile uint16_t counter = ?;

ISR(TCA0_OVF_vect){
    BUZZER_PORT.OUTTGL = (1<<BUZZER_PIN_bp);
```

```

    TCA0.SINGLE.INTFLAGS |= TCA_SINGLE_OVF_bm;
}
ISR(TCA1_OVF_vect){
    counter = (counter + ?)%?;
    TCA0.SINGLE.PERBUF = ? - counter; // ?
    TCA1.SINGLE.INTFLAGS |= TCA_SINGLE_OVF_bm;
}
int main(void)
{
    usart_usb_init();
    BUZZER_PORT.DIRSET = (1<<BUZZER_PIN_bp);
    TCA0.SINGLE.CTRLA |= TCA_SINGLE_ENABLE_bm | ?;
    TCA0.SINGLE.INTCTRL |= TCA_SINGLE_OVF_bm;
    TCA0.SINGLE.PERBUF = ?; // ?
    TCA1.SINGLE.CTRLA |= TCA_SINGLE_ENABLE_bm | ?;
    TCA1.SINGLE.INTCTRL |= TCA_SINGLE_OVF_bm;
    TCA1.SINGLE.PERBUF = ?; //?
    sei(); // ?
    while(1){
    }
}

```

b) 6%

Forklar hva som skjer i de kommenterte kodelinjene og den generelle funksjonaliteten til funksjonen. Illustrer hvordan funksjonen brukes i kode.

```

uint8_t get_ADC_filtered(uint16_t oldVals, uint16_t *res, uint8_t filterSize)
{ // 1
    if(filterSize == 0)
        return 1;
    while(!(ADC0.INTFLAGS & ADC_RESRDY_bm)); // 2
    oldVals[filterSize-1] = ADC0.RES; // 3
    uint16_t tmp = 0;
    for(uint8_t i = 0; i<filterSize; i++){
        tmp += oldVals[i]; // 4
    }
    *res=tmp/filterSize;
    return 0; // 5
}

```

c) 5%

Hva kan man si om signalene på banene/lederne A og B ut i fra illustrasjonen i figur 2?

(Figur 2: Illustrasjon av krets med mikrokontroller, motstander og tre sensorer)

Oppgave 3: Programmering og implementasjon

I denne oppgaven skal du vise at du kan lage egen kode, tilpasse kode til andre formål og beskrive hvordan mikrokontrolleren kan brukes til å løse problemer. Bruk gjerne pseudokode dersom du ikke husker kodeordene ordrett. Du kan ta utgangspunkt i at koden som følger med i oppgavesettet kan brukes som om det

var c-filer med tilhørende h-filer. Husk at du kan få poeng selv om bare deler av oppgaven er fullført.

- a) **10%** Skriv et program som inkrementer et binært tall visualisert med lysdiode D0-2 (3 stykk) ca. hvert sekund. Tallet starter på 0b000, inkrementeres helt til 0b111, og starter på nytt. Dersom knappene SW1–3 trykkes inn med samme konfigurasjon som lysene, skal mikrokontrolleren sende meldingen “suksess!” til en datamaskin ved å bruke USART. Forklar hvordan implementasjonen hadde sett ut dersom det også skulle benyttes søvnmodus. (Tips: CCL må deaktiveres mens det gjøres endringer).
- b) **15%** Lag et program som foretar ADC lesinger med en frekvens på 10 Hz (relativt nøyaktig) fra meldingen “<n1>” mottas, til meldingen “<n0>” mottas via USART, der n er et tall som symboliserer AINn. Dersom det går mer enn 10 sekunder før “<n0>” mottas, skal også avlesingene slutte. Deretter skal alle verdiene returneres til datamaskinen med USART.
- c) **15%** En vifte med PWM-styring og takometer brukes til å kjøle ned en komponent med en analog temperatursensor. Hastigheten på viften justeres mellom 1ms og 2ms med PWM-frekvens på 50Hz. Temperatursensoren er koblet på PD4 og gir spenninger mellom 0V og 2.5V der relasjonen mellom temperatur og spenning er gitt av: $T \approx (30 + 20 \times V_{sens})^\circ\text{C}$. Lag et program som justerer blåsing mellom minimum og maksimum hastighet fra $T = 50^\circ\text{C}$ til $T = 80^\circ\text{C}$. Foreslå en måte å måle om hastigheten på vifta samsvarer med pådraget/PWM-verdien.

Lykke til!

1 Kode segmenter

sleep.c

```
/*
 * GccApplication6.c
 *
 * Created: 10/28/2024 12:19:49 PM
 * Author : jensc
 */

#define F_CPU 4000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>
#include <avr/cpufunc.h>
#include <util/delay.h>
#include "adc.h"
#include "usart.h"

// Define pins for buttons on PORTD
```

```

#define BUTTON_SW1_PIN 0 // PD0
#define BUTTON_SW2_PIN 1 // PD1
#define BUTTON_SW3_PIN 2 // PD2

volatile uint8_t rtc_counter = 0;

void XOSC32K_init()
{
    ccp_write_io((void *)&(CLKCTRL.XOSC32KCTRLA), CLKCTRL_ENABLE_bm);
}

void rtc_init() {
    XOSC32K_init();
    RTC.CLKSEL = RTC_CLKSEL_XOSC32K_gc;
    RTC.CTRLA |= RTC_RTCEN_bm | RTC_PRESCALER_DIV1_gc;
    RTC.PER = 32768; //32720 div64 (32768 / 64) * 60
    RTC.INTCTRL |= RTC_OVF_bm;
}

void enter_sleep_mode() {
    set_sleep_mode(SLEEP_MODE_STANDBY);
    sleep_cpu();
}

ISR(RTC_CNT_vect) {
    RTC.INTFLAGS = RTC_OVF_bm; // Clear the overflow flag.
    // Timer overflow ISR - triggers measurement and transmission every
    // minute

    rtc_counter++;

    // Check if the counter has reached 60 (1 minute)
    if (rtc_counter >= 60) {
        rtc_counter = 0; // Reset counter
        take_measurement_and_send(); // Take measurement and send
        data
    }
    sleep_cpu();
}

ISR(PORTD_PORT_vect) {
    // Simple debounce by checking the button states after a short delay
    _delay_ms(100); //Delay to let the signal settle

    if ((PORTD.IN & (1 << BUTTON_SW1_PIN)) &&
        (PORTD.IN & (1 << BUTTON_SW2_PIN)) &&
        (PORTD.IN & (1 << BUTTON_SW3_PIN))) {
        take_measurement_and_send();
    }

    _delay_ms(20);

    // Clear interrupt flags for all buttons
    PORTD.INTFLAGS = (1 << BUTTON_SW1_PIN) | (1 << BUTTON_SW2_PIN) | (1
        << BUTTON_SW3_PIN);
    sleep_cpu();
}

```

```

}

void take_measurement_and_send() {
    //ADC_Start();
    uint16_t adc_result = adc_get_result();
    char message[20];
    snprintf(message, sizeof(message), "ADC: %u\n", adc_result);
    usart_usb_transmit_char_array(1, message, strlen(message));
}

int main(void) {
    usart_usb_init();
    ADC_Init();

    // Set button pins as inputs with pull-up resistors enabled
    PORTD.DIRCLR = (1 << BUTTON_SW1_PIN) | (1 << BUTTON_SW2_PIN) | (1 <<
        BUTTON_SW3_PIN);
    PORTD.PIN0CTRL = PORT_PULLUPEN_bm | PORT_ISC_RISING_gc;
    PORTD.PIN1CTRL = PORT_PULLUPEN_bm | PORT_ISC_RISING_gc;
    PORTD.PIN2CTRL = PORT_PULLUPEN_bm | PORT_ISC_RISING_gc;

    // Initialize timer for 1 minute intervals
    rtc_init();

    // Enable global interrupts
    sei();

    ADC_Start();
    enter_sleep_mode();

    while (1) {
        //enter_sleep_mode();
    }
}

```

adc.c

```

#include "adc.h"
#include <avr/io.h>

#define SCALING_FACTOR 4096

void ADC_Init(void) {
    VREF.ADCOREF = VREF_REFSEL_2V048_gc | VREF_ALWAYS_ON_bm;

    // Set ADC clock, sample length, and initialization delay
    ADC0.CTRLA = ADC_ENABLE_bm | ADC_FREERUN_bm | ADC_RESSEL_12BIT_gc; //
    // Enable ADC, freerunning makes start conversion only necessary in
    // init
    ADC0.CTRLB = ADC_PRESC_DIV16_gc; // Set prescaler for 250kHz
    ADC0.MUXPOS = ADC_MUXPOS_AIN3_gc;
}

// Start the conversion
void ADC_Start(void) {

```

```

        ADC0.COMMAND = ADC_STCONV_bm;
    }

    uint16_t adc_get_result(){
        while(!ADC0.INTFLAGS & ADC_RESRDY_bm)
            ;
        return ADC0.RES;
    }

```

adc.h

```

#ifndef ADC_H_
#define ADC_H_

#include <avr/io.h>
#include <stdbool.h>

void ADC_Init(void);
void ADC_Start(void);
uint16_t ADC_GetResult(void);
#endif /* ADC_H_ */

```

usart.c

```

/*
 * usart.c
 *
 * Created: 24.08.2022 14:14:12
 * Author: pol022
 */

#include <avr/io.h>
#include "usart.h"
#include <avr/interrupt.h>

void usart_usb_init(){
    /* Necessary configuration*/
    USART3.BAUD = 1667; // Baudrate 9600 // F_CPU = 16M
    USART3.CTRLB |= USART_RXEN_bm | USART_TXEN_bm |
        USART_RXMODE_NORMAL_gc; //enable RX TX
    PORTB.DIR |= (1<<TX_PIN_bp);
    /* interrupt on RX*/
    USART3.CTRLA |= USART_RXCIE_bm;

    /* interrupt on TX */
    //USART3.CTRLA |= USART_TXCIE_bm;

    /* printf */
    stdout = &new_std_out; // address to struct
}

void usart_usb_transmit(char c){
    while(!(USART3.STATUS & USART_DREIF_bm)){

```

```

        ;
    }
    USART3.TXDATAL = c;
}

/* transmitting*/
void usart_usb_transmit_char_array(uint8_t addEndLine, char string[], uint8_t
length){
    for(uint8_t i = 0; i<length; i++){
        usart_usb_transmit(string[i]);
    }
    if(addEndLine)
        usart_usb_transmit('\n');
}

/* rx-polling*/
char usart_usb_receive(){
    while(!(USART3.STATUS & USART_RXCIF_bm))// not unread data in
        receiver
        ;
    return USART3.RXDATAL;
}

/* rx-interrupt with some handling.*/
ISR(USART3_RXC_vect){
    char tmp = USART3.RXDATAL;
    switch (tmp)
    {
        case '<':
            marker_pos = 0;
            break;
        case '>':
            usb_message_ready = 1;
            usb_message[marker_pos] = 0;
            break;
        default:
            usb_message[marker_pos] = tmp;
            marker_pos++;
    }
}

static uint8_t usart_usb_transmit_printf(char c, FILE *stream){
    usart_usb_transmit(c);
    return 0;
}

/* Generic usart-files */
void usart_init(USART_t *usart_nr, PORT_t *usart_port, uint8_t tx_bp, uint8_t
rx_bp){
    /* Necessary configuration*/
    //PORTMUX.USARTROUTEA |= PORTMUX_USART2_ALT1_gc;// non generic
    usart_nr->BAUD = 1667; // Baudrate 9600
    usart_nr->CTRLB |= USART_RXEN_bm | USART_TXEN_bm |
        USART_RXMODE_NORMAL_gc; //enable RX TX

```



```

    usart_port->DIR |= (1<<tx_bp);
    usart_port->PINCONFIG |= PORT_PULLUPEN_bm;
    usart_port->PINCTRLSET |= (1<<rx_bp);
    // RX-handling in separate function.
    //usart_port->CTRLA |= USART_RXCIE_bm; // interrupt on RX
}

void usart_transmit_char(USART_t *usart_nr, char c){
    while(!(usart_nr->STATUS & USART_DREIF_bm)){
        ;
    }
    usart_nr->TXDATA = c;
}

void usart_transmit_string(USART_t *usart_nr, char *string,
    usart_char_transmitter transmit_char){
    for(uint8_t i = 0; string[i] != '\0'; i++)
        transmit_char(usart_nr, string[i]);
}

// example with USART2
ISR(USART2_RXC_vect){
    char tmp = USART2.RXDATA;
    switch (tmp)
    {
        case '{':
            u2_marker_pos = 0;
            break;
        case '}':
            u2_message_ready = 1;
            u2_message[marker_pos] = 0;
            break;
        default:
            u2_message[marker_pos] = tmp;
            u2_marker_pos++;
    }
}

```

usart.h

```

/*
 * usart.h
 *
 * Created: 24.08.2022 14:14:25
 * Author: pol022
 */

#ifndef USART_H_
#define USART_H_

#include <avr/io.h>
#define TX_PIN_bp 0
#define RX_PIN_bp 1
#include <stdio.h>

char usb_message[256];

```

```

volatile uint8_t marker_pos;
volatile uint8_t usb_message_ready;

volatile uint8_t u2_marker_pos;
volatile uint8_t u2_message_ready;
char u2_message[256];

void usart_usb_init();

void usart_usb_transmit(char c);
char usart_usb_receive();

/* printing an array of chars*/
void usart_usb_transmit_char_array(uint8_t addEndLine, char string[], uint8_t
    length);

/* preparing for printf */
static uint8_t usart_usb_transmit_printf(char c, FILE *stream);
static FILE new_std_out = FDEV_SETUP_STREAM(usart_usb_transmit_printf, NULL,
    _FDEV_SETUP_WRITE);

/* generic usart files */
void usart_init(USART_t *usart_nr, PORT_t *usart_port, uint8_t tx_bp, uint8_t
    rx_bp);
void usart_transmit_char(USART_t *usart_nr, char c);
typedef void (*usart_char_transmitter)(USART_t *usart_nr, char c);
void usart_transmit_string(USART_t *usart_nr, char *string,
    usart_char_transmitter transmit_char);

#endif /* USART_H_ */

```